Optimizing Hospital Medicine Stocks Using Multi-Agent Predictive Systems

Table of Contents

1. Application and Workflow Selection 4

1.1 Introduction and Motivation 4

1.2 Explanation of the Selected Workflow 4

1.3 Objectives of the Intelligent System.....6

1.4 Impact of the Project if Developed.....6

2. Selection of AI Technologies 7

2.1 Similar Systems.....7

2.2 Innovative Approaches 7

2.3 Justification for Selected AI Methods 8

3. Simulations & Development Setup 8

3.1 Simulation Environment and Setup Parameters.....8

3.2 Input Data Preparation and Cleaning (Agent 1).....9

3.3 Feature Engineering (Agent 2).....10

3.4 Model Training (Agent 3) 11

3.5 Model Evaluation (Agent 4) 12

3.6 Reporting (Agent 5).....13

4. Results & Discussion 15

4.1 Exploratory Data Analysis 16

4.2 Feature Engineering Insights 18

4.3 Model Performance 19

4.4 System Impact and Benefits.....20

5. Conclusion 20

Reference.....22

Appendix 25

List of Figures:

Figure 1 Simulation Environment and Setup Parameters 9
Figure 2 Merge dataset overview.....10
Figure 3 Feature Engineering 11
Figure 4 Model training.....12
Figure 5 Model Evaluation.....13
Figure 6 Feature selection 15
Figure 7 EDA.....18
Figure 8 Feature Engineering Insights 19
Figure 9 Model Performance.....20

1. Application and Workflow Selection

1.1 Introduction and Motivation

To ensure continuity in the treatment of patients, the hospitals should aim to have the medicine in time. Any disruption, lack of, or mismanagement in medicine stores can undermine their quality of treatment, raise costs of running the business, and pose serious administrative congestions. The conventional manual inventory management systems are normally slow, reactive, and prone to human error, and this can either lead to stock outage or wastage since the medicine is expired. These inefficiencies have the ability to impact the performance of the hospital and directly affect patient health(Munavalli*et al.*, 2020).

These difficulties require the invention of intelligent systems that would analyze the consumption patterns, predict future needs of medicine, and automate purchase orders. Multi-agent AI systems are, in particular, effective since they allocate particular tasks to focus on intelligent agents and allow such operations to be modular, scaled, and efficient. This guarantees that there is accuracy in the forecasting, minimizes human error, and makes the process of acquiring medicine easier(Kusne and McDannald, 2023). The implementation of such a system is a significant move toward modernization of managing the hospital stocks and maximizing its operation efficiency.

1.2 Explanation of the Selected Workflow

[redacted]

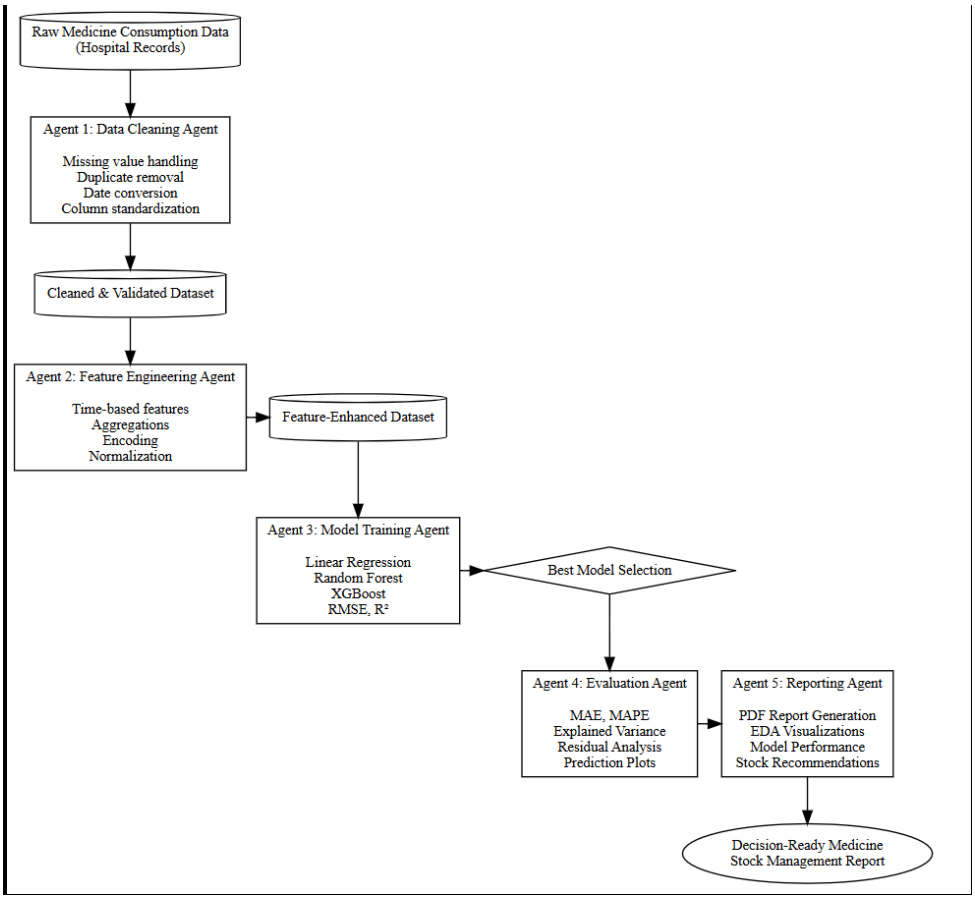
[redacted]

[redacted]

[redacted]

[redacted]

[redacted]



[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```python
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
import pandas as pd
```

```python
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np
```

```python
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 4. Initialize models
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from xgboost import XGBRegressor
```

```python
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score, explained_variance_score
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import os
```

```python
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.ensemble import RandomForestRegressor
from xgboost import XGBRegressor
from sklearn.linear_model import LinearRegression

# Use X_final and y_final from feature engineering step
X_train, X_test, y_train, y_test = train_test_split(X_final, y_final, test_size=0.2, random_state=42)
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```
sales_daily['frequency'] = 'daily'
sales_hourly['frequency'] = 'hourly'
sales_weekly['frequency'] = 'weekly'
sales_monthly['frequency'] = 'monthly'

master_df = pd.concat([
    sales_daily,
    sales_hourly,
    sales_weekly,
    sales_monthly
], ignore_index=True)

master_df.head()
```

	datum	M01AB	M01AE	N02BA	N02BE	N05B	N05C	R03	R06	Year	Month	Hour	Weekday Name	frequency
0	2014-01-02	0.0	3.67	3.4	32.40	7.0	0.0	0.0	2.0	2014.0	1.0	248.0	Thursday	daily
1	2014-01-03	8.0	4.00	4.4	50.60	16.0	0.0	20.0	4.0	2014.0	1.0	276.0	Friday	daily
2	2014-01-04	2.0	1.00	6.5	61.85	10.0	0.0	9.0	1.0	2014.0	1.0	276.0	Saturday	daily
3	2014-01-05	4.0	3.00	7.0	41.10	8.0	0.0	3.0	0.0	2014.0	1.0	276.0	Sunday	daily
4	2014-01-06	5.0	1.00	4.5	21.70	16.0	2.0	6.0	2.0	2014.0	1.0	276.0	Monday	daily

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```python
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
import pandas as pd

# ------------------- 1. Handle missing values -------------------
# Fill numeric NaNs with median
master_df[numeric_cols] = master_df[numeric_cols].fillna(master_df[numeric_cols].median())

# Fill categorical NaNs with mode
for col in categorical_cols:
    master_df[col] = master_df[col].fillna(master_df[col].mode()[0])

# ------------------- 2. Feature Creation -------------------
# Example: time features if 'datum' exists
if 'datum' in master_df.columns:
    master_df['datum'] = pd.to_datetime(master_df['datum'])
    master_df['Year'] = master_df['datum'].dt.year
    master_df['Month'] = master_df['datum'].dt.month
    master_df['Day'] = master_df['datum'].dt.day
    master_df['Weekday'] = master_df['datum'].dt.weekday

# ------------------- 3. Encode categorical variables -------------------
categorical_cols = [col for col in categorical_cols if col in master_df.columns]
if categorical_cols:
    ohe = OneHotEncoder(sparse_output=False, drop='first')  # fixed parameter
    cat_encoded = ohe.fit_transform(master_df[categorical_cols])
    cat_encoded_df = pd.DataFrame(cat_encoded, columns=ohe.get_feature_names_out(categorical_cols))
    master_df = pd.concat([master_df.drop(columns=categorical_cols), cat_encoded_df], axis=1)

# ------------------- 4. Scale numeric features -------------------
scaler = StandardScaler()
master_df[numeric_cols] = scaler.fit_transform(master_df[numeric_cols])

print("Feature engineering completed. Dataset is ready for model training.")
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```python
lr = LinearRegression()
rf = RandomForestRegressor(random_state=42)
xgb = XGBRegressor(random_state=42, objective='reg:squarederror')

# 5. Train models
lr.fit(X_train, y_train)
rf.fit(X_train, y_train)
xgb.fit(X_train, y_train)

# 6. Evaluate models
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np

def evaluate_model(y_true, y_pred, model_name):
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    r2 = r2_score(y_true, y_pred)
    print(f"{model_name} → RMSE: {rmse:.4f}, R2: {r2:.4f}")

y_pred_lr = lr.predict(X_test)
y_pred_rf = rf.predict(X_test)
y_pred_xgb = xgb.predict(X_test)

evaluate_model(y_test, y_pred_lr, "Linear Regression")
evaluate_model(y_test, y_pred_rf, "Random Forest")
evaluate_model(y_test, y_pred_xgb, "XGBoost")
```

```
Linear Regression → RMSE: 0.2208, R2: 0.9537
Random Forest → RMSE: 0.1842, R2: 0.9678
XGBoost → RMSE: 0.1996, R2: 0.9622
```

[redacted]

[redacted]

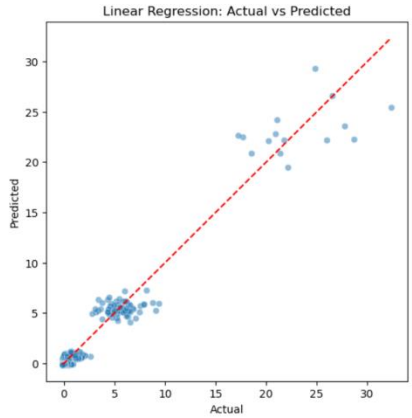
[redacted]

[redacted]

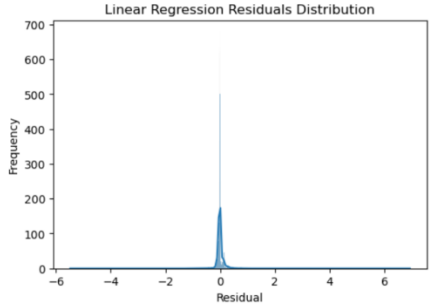
[redacted]

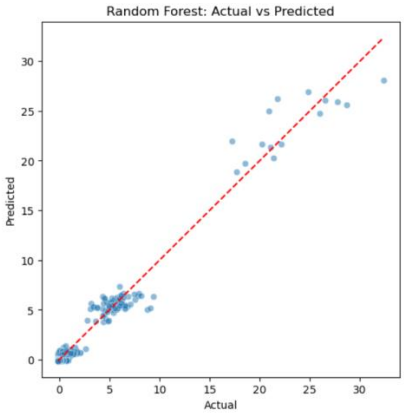
[redacted]

[redacted]

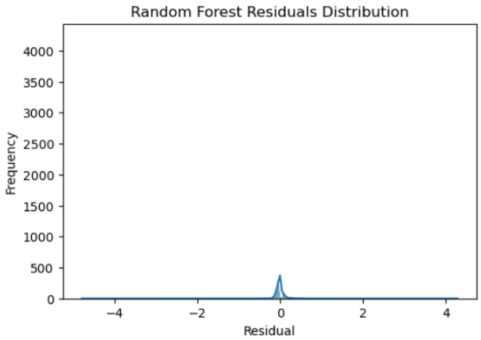


[redacted]

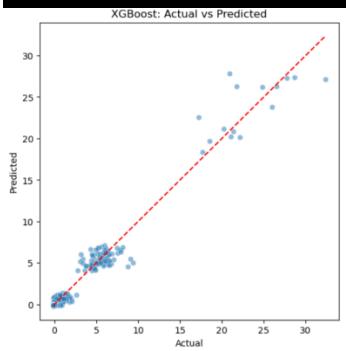


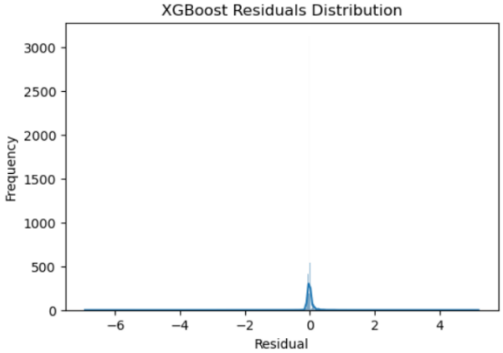


[redacted]



[redacted]





[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

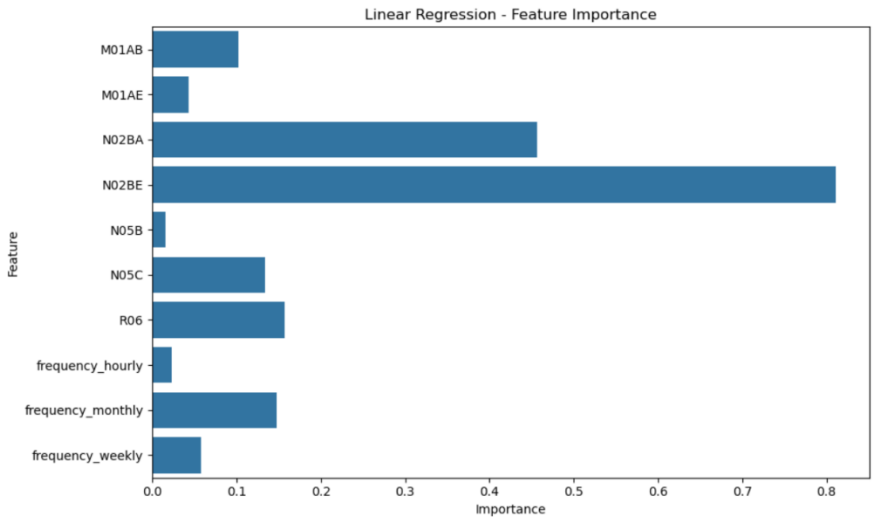
[redacted]

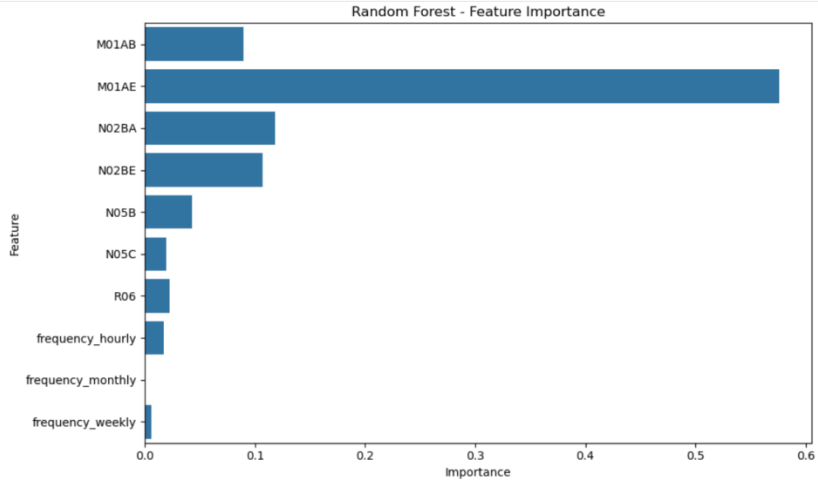
[redacted]

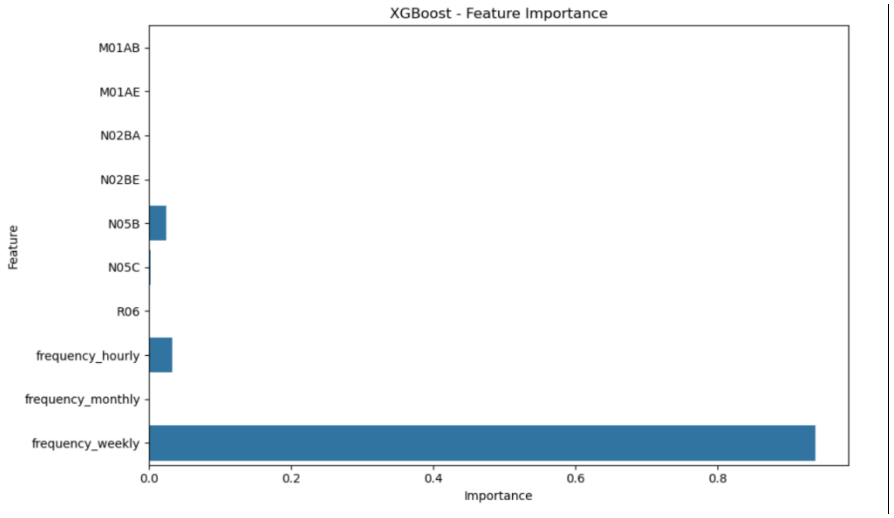
[redacted]

[redacted]

[redacted]







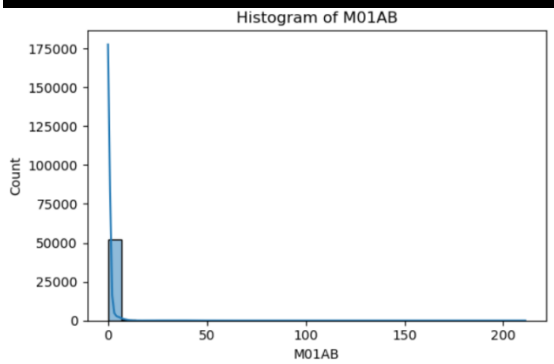
[redacted]

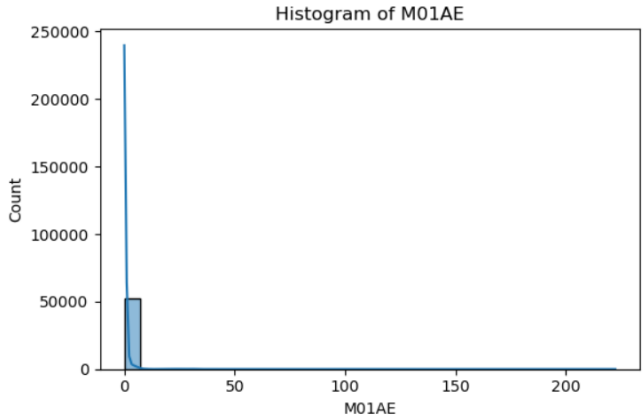
[redacted]

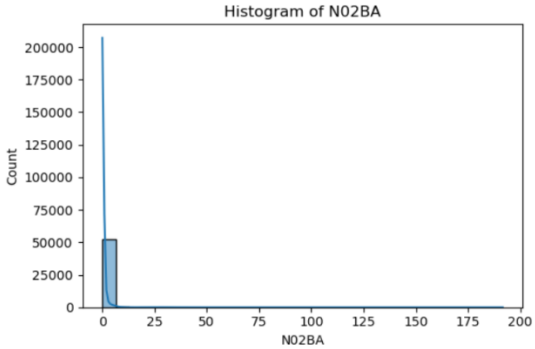
[redacted]

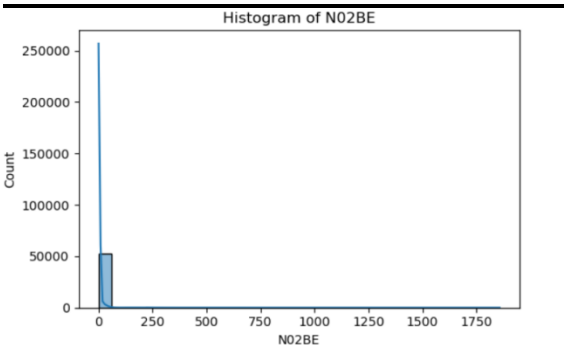
[redacted]

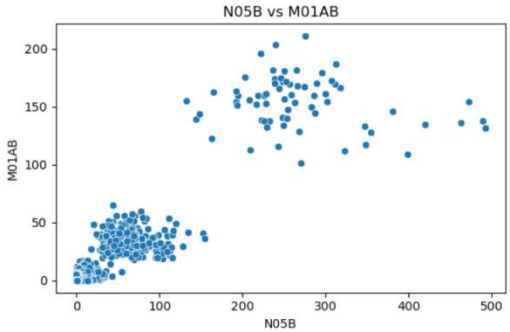
[redacted]



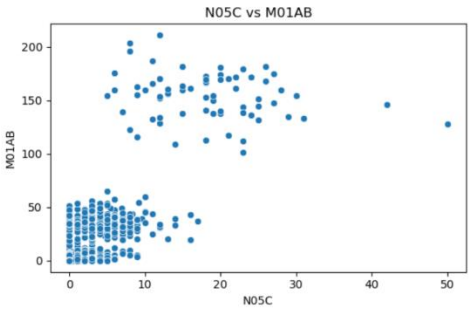




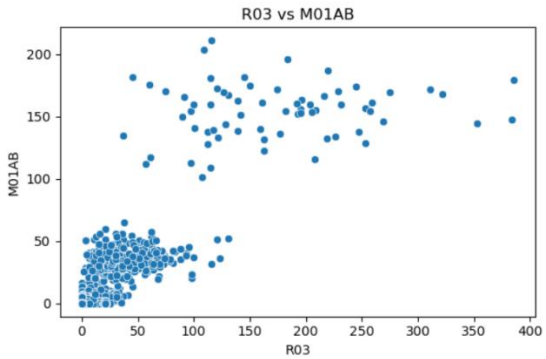


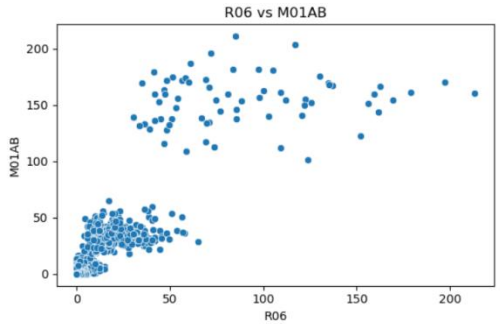


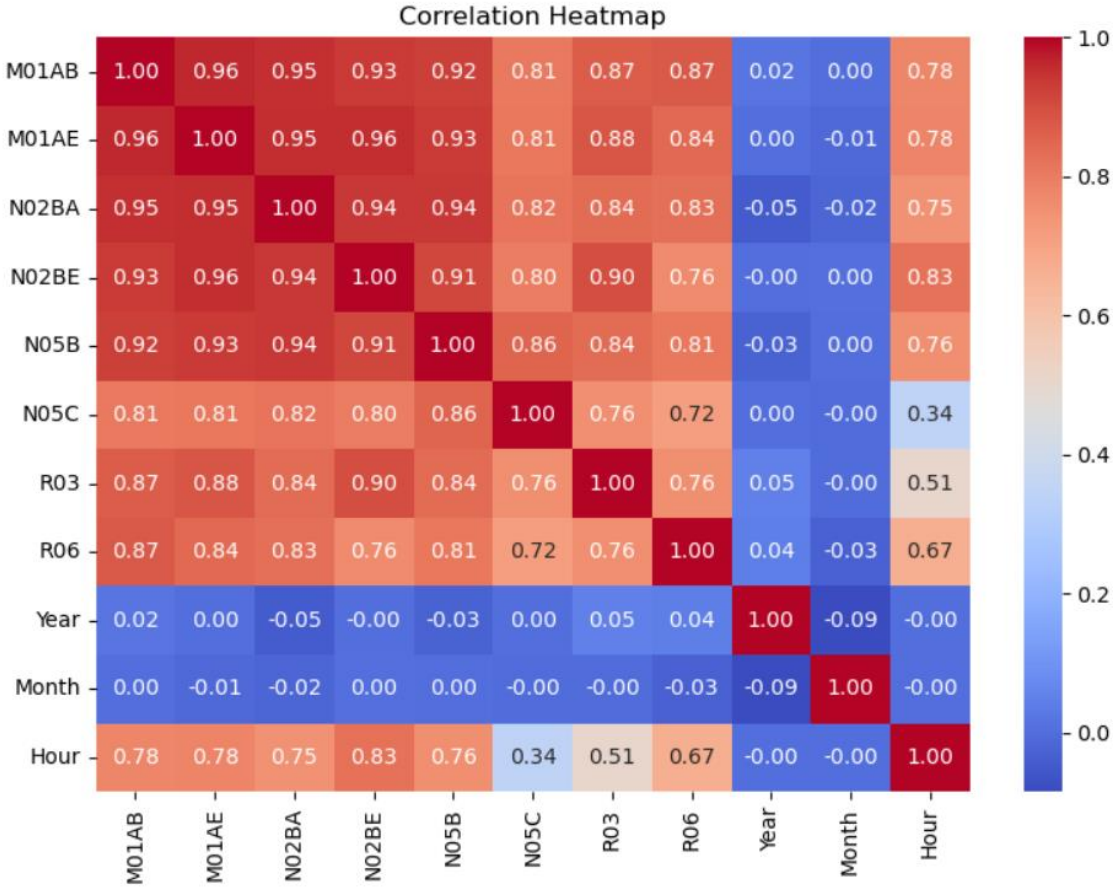
[redacted]

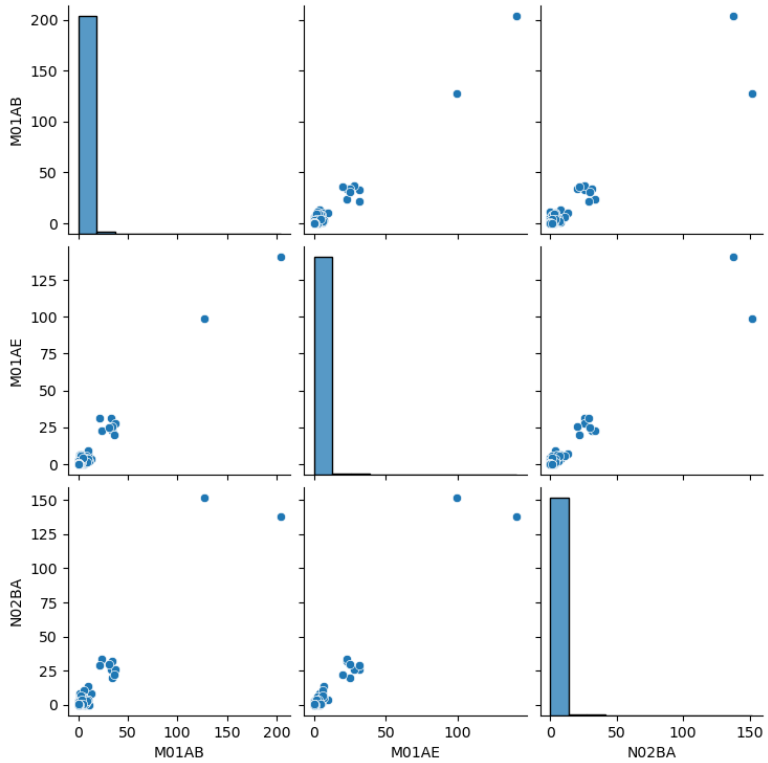


[redacted]


R03 vs M01AB
M01AB
R03
0
50
100
150
200
250
300
350
400


R06 vs M01AB
M01AB
R06
0
50
100
150
200


Correlation Heatmap
M01AB 1.00 0.96 0.95 0.93 0.92 0.81 0.87 0.87 0.02 0.00 0.78
M01AE 0.96 1.00 0.95 0.96 0.93 0.81 0.88 0.84 0.00 -0.01 0.78
N02BA 0.95 0.95 1.00 0.94 0.94 0.82 0.84 0.83 -0.05 -0.02 0.75
N02BE 0.93 0.96 0.94 1.00 0.91 0.80 0.90 0.76 -0.00 0.00 0.83
N05B 0.92 0.93 0.94 0.91 1.00 0.86 0.84 0.81 -0.03 0.00 0.76
N05C 0.81 0.81 0.82 0.80 0.86 1.00 0.76 0.72 0.00 -0.00 0.34
R03 0.87 0.88 0.84 0.90 0.84 0.76 1.00 0.76 0.05 -0.00 0.51
R06 0.87 0.84 0.83 0.76 0.81 0.72 0.76 1.00 0.04 -0.03 0.67
Year 0.02 0.00 -0.05 -0.00 -0.03 0.00 0.05 0.04 1.00 -0.09 -0.00
Month 0.00 -0.01 -0.02 0.00 0.00 -0.00 -0.00 -0.03 -0.09 1.00 -0.00
Hour 0.78 0.78 0.75 0.83 0.76 0.34 0.51 0.67 -0.00 -0.00 1.00
M01AB M01AE N02BA N02BE N05B N05C R03 R06 Year Month Hour
1.0
0.8
0.6
0.4
0.2
0.0



[redacted]

[redacted]

[redacted]

```
Dataset shape after feature engineering: (53010, 23)

Dataset statistics:
```

	datum	M01AB	M01AE	N02BA	N02BE	N05B	N05C	R03	R06	Year	...	We
count	53010	5.301000e+04	5.301000e+04	5.301000e+04	53010.000000	5.301000e+04	5.301000e+04	53010.000000	5.301000e+04	5.301000e+04	...	53010.0
mean	2016-11-20 02:12:44.006791168	-5.147112e-17	1.715704e-17	3.431408e-17	0.000000	-1.715704e-17	8.578520e-18	0.000000	2.573556e-17	1.554428e-14	...	3.0
min	2014-01-02 00:00:00	-1.270618e-01	-1.261404e-01	-1.255372e-01	-0.122016	-1.238271e-01	-1.077736e-01	-0.112635	-1.141877e-01	-1.442796e+00	...	0.0
25%	2015-06-12 17:15:00	-1.270618e-01	-1.261404e-01	-1.255372e-01	-0.122016	-1.238271e-01	-1.077736e-01	-0.112635	-1.141877e-01	-8.420040e-01	...	1.0
50%	2016-11-20 01:30:00	-1.270618e-01	-1.261404e-01	-1.255372e-01	-0.122016	-1.238271e-01	-1.077736e-01	-0.112635	-1.141877e-01	-2.412121e-01	...	3.0
75%	2018-04-30 10:45:00	-1.270618e-01	-1.261404e-01	-1.255372e-01	-0.070579	-1.238271e-01	-1.077736e-01	-0.112635	-1.141877e-01	9.603717e-01	...	5.0
max	2019-10-31 00:00:00	3.348997e+01	4.524466e+01	3.902463e+01	47.632576	4.335068e+01	5.703588e+01	49.383266	5.275826e+01	1.561164e+00	...	6.0
std	NaN	1.000009e+00	1.000009e+00	1.000009e+00	1.000009	1.000009e+00	1.000009e+00	1.000009	1.000009e+00	1.000009e+00	...	2.0

8 rows × 23 columns

```
Numeric columns after scaling:
      M01AB     M01AE     N02BA     N02BE      N05B      N05C       R03  \
0 -0.127062  0.622725  0.569194  0.711265  0.494713 -0.107774 -0.112635
1  1.146733  0.690061  0.773527  1.179343  1.289978 -0.107774  2.451920
2  0.191387  0.077910  1.202626  1.468676  0.759801 -0.107774  1.041415
3  0.509835  0.486011  1.304792  0.935017  0.583075 -0.107774  0.272049
4  0.669060  0.077910  0.793960  0.436077  1.289978  2.177973  0.656732

        R06      Year     Month      Hour
0  0.382174 -1.442796 -1.578653  4.338241
1  0.878536 -1.442796 -1.578653  4.875746
2  0.133993 -1.442796 -1.578653  4.875746
3 -0.114188 -1.442796 -1.578653  4.875746
4  0.382174 -1.442796 -1.578653  4.875746

Newly created time features:
0   -1.442796
1   -1.442796
2   -1.442796
3   -1.442796
4   -1.442796
Name: Year, dtype: float64
0   -1.578653
1   -1.578653
2   -1.578653
3   -1.578653
4   -1.578653
Name: Month, dtype: float64
0    2
1    3
2    4
3    5
4    6
Name: Day, dtype: int32
0    3
```

[redacted]

[redacted]

```
                Model      RMSE       MAE       MAPE        R2  \
0  Linear Regression  0.220765  0.071402  63.882342  0.953694
1      Random Forest  0.184184  0.068533  59.639992  0.967768
2            XGBoost  0.199585  0.068011  60.386749  0.962152

   Explained Variance
0            0.953704
1            0.967789
2            0.962155

Best Model → Random Forest (based on lowest RMSE)
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

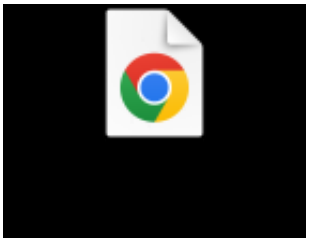
[redacted]

[redacted]

[redacted]

[redacted]

[redacted]



[redacted]